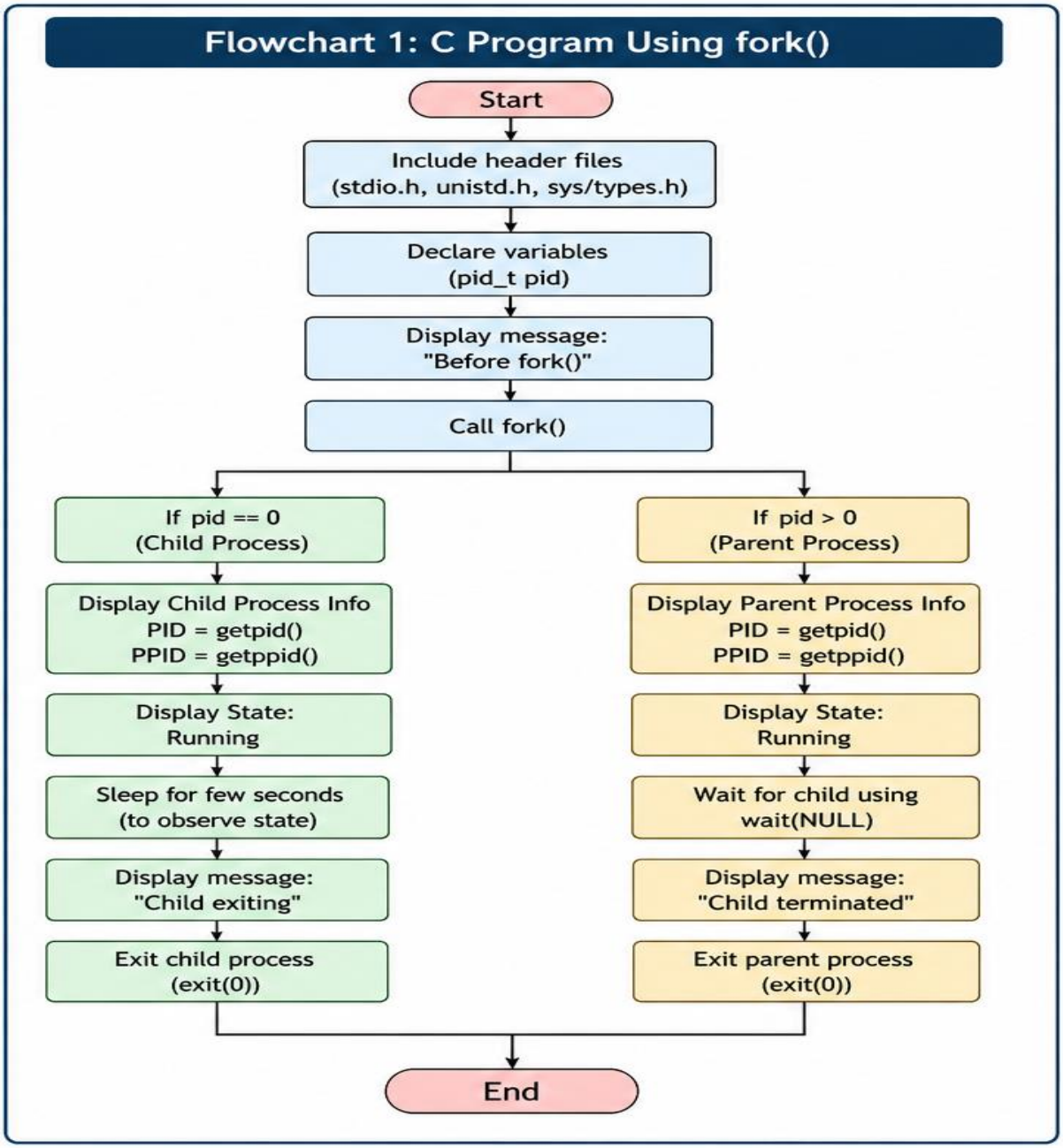
Task 3: Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Aim

To develop a C program using fork() to create a parent and child process and display their PID, PPID and process states.

FlowChart



Program

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
   pid_t pid;

   printf("Before fork():\n");
   printf("PID  = %d\n", getpid());
   printf("PPID = %d\n\n", getppid());

   pid = fork();

   if (pid < 0) {
      perror("fork failed");
      return 1;
   }

   if (pid == 0) {
      // Child process
      printf("Child Process:\n");
      printf("PID  = %d\n", getpid());
      printf("PPID = %d\n", getppid());
      printf("State: Running\n\n");
```

```
        sleep(5);

        printf("Child Process finishing...\n");
    }
    else {
        // Parent process
        printf("Parent Process:\n");
        printf("PID  = %d\n", getpid());
        printf("Child PID = %d\n", pid);
        printf("State: Running\n\n");

        wait(NULL);

        printf("Parent: Child process terminated.\n");
    }

    return 0;
}
```

Output

```
amrutha@blue:~$ mkdir -p ~/OSSP/Week3
amrutha@blue:~$ cd ~/OSSP/Week3
amrutha@blue:~/OSSP/Week3$ pwd
/home/amrutha/OSSP/Week3
amrutha@blue:~/OSSP/Week3$ nano fork_process.c
amrutha@blue:~/OSSP/Week3$ gcc fork_process.c -o fork_process
amrutha@blue:~/OSSP/Week3$ ls
fork_process  fork_process.c
amrutha@blue:~/OSSP/Week3$ ./fork_process
Before fork():
PID  = 649
PPID = 322

Parent Process:
PID  = 649
Child PID = 650
State: Running

Child Process:
PID  = 650
PPID = 649
State: Running

Child Process finishing...
Parent: Child process terminated.
amrutha@blue:~/OSSP/Week3$
```

Observation:

The fork() function successfully created a child process. The parent and child processes had different PIDs, while the child's PPID matched the parent's PID.

Result

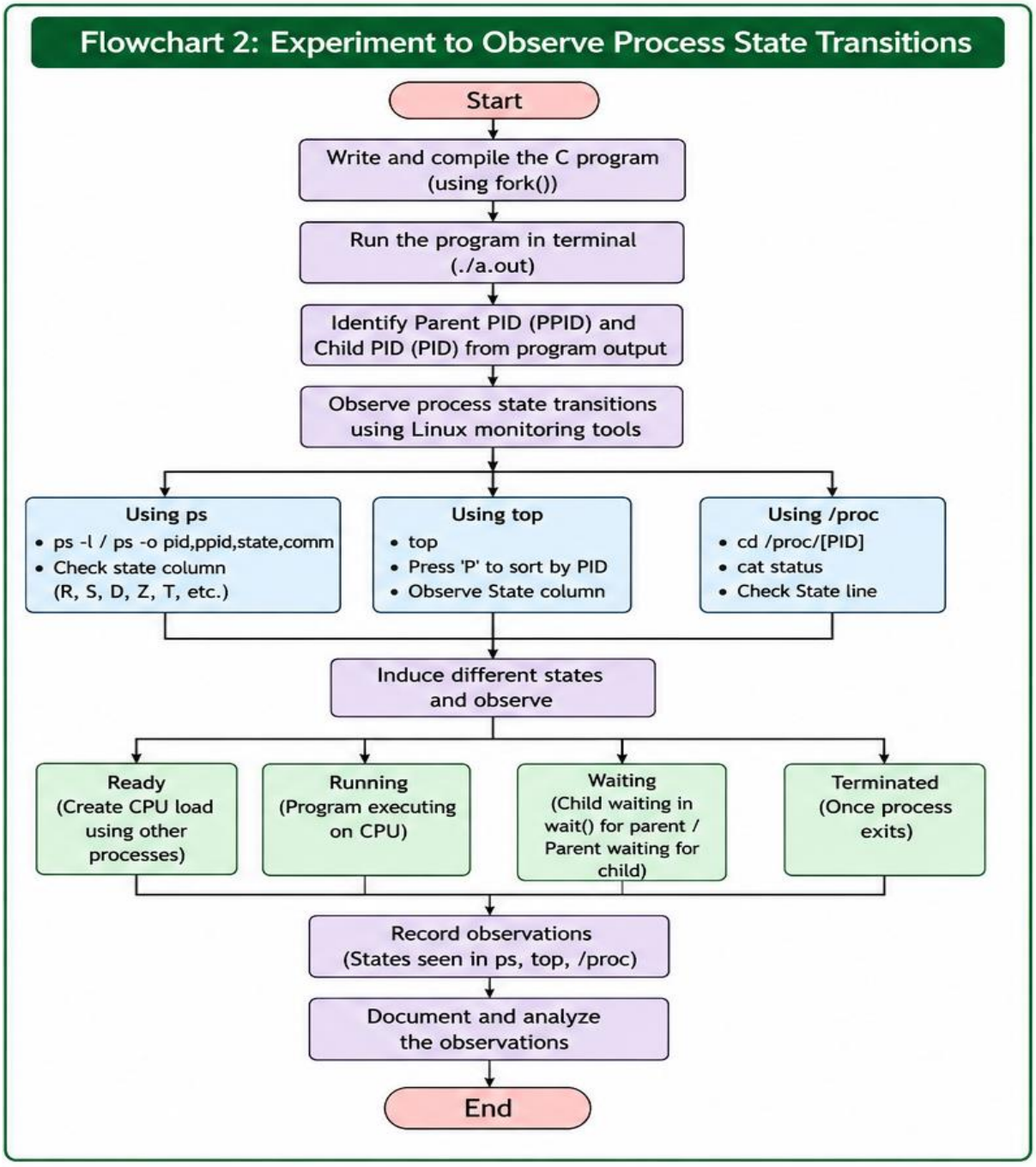
The parent and child processes were successfully created using fork(), and their PID, PPID and states were displayed.

Task 4: Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

Aim

To observe process state transitions using Linux commands such as ps, top and /proc.

Flow Chart:



Program

```
#include <stdio.h>
#include <unistd.h>

int main() {
   printf("Process started!\n");
   printf("PID = %d\n", getpid());

   while (1) {
      sleep(2);
   }

   return 0;
}
```

Procedure

1. Compile and run the program.
2. Note the PID displayed.
3. Use ps to check the process details.
4. Use /proc/<PID>/status to check the process state.
5. Use top to monitor the process.
6. Stop the process using Ctrl+C.

OUTPUT

```
amrutha@blue:~/OSSP/Week3$ nano process_monitor.c
amrutha@blue:~/OSSP/Week3$ gcc process_monitor.c -o process_monitor
amrutha@blue:~/OSSP/Week3$ ./process_monitor
Process started!
PID = 998
^C
amrutha@blue:~/OSSP/Week3$
```

```
amrutha@blue:~$ ps -p 998 -o pid,ppid,state,stat,cmd
    PID    PPID S STAT CMD
    998     322 S S+   ./process_monitor
amrutha@blue:~$ cat /proc/998/status
Name:   process_monitor
Umask:  0022
State:  S (sleeping)
Tgid:   998
Ngid:   0
Pid:    998
PPid:   322
TracerPid:      0
Uid:    1000    1000    1000    1000
Gid:    1000    1000    1000    1000
FDSize: 256
Groups: 4 24 27 30 46 100 1000
NStgid: 998
NSpid:  998
NSpgid: 998
NSsid:  322
Kthread:        0
VmPeak:     2768 kB
VmSize:     2768 kB
VmLck:         0 kB
VmPin:         0 kB
VmHWM:      1944 kB
VmRSS:      1944 kB
RssAnon:              96 kB
RssFile:            1848 kB
RssShmem:              0 kB
```

Observations

1. Using ps

```
amrutha@blue:~$ ps -p 998 -o pid,ppid,state,stat,cmd
    PID    PPID S STAT CMD
    998     322 S S+   ./process_monitor
```

2. Using /proc

```
amrutha@blue:~$ cat /proc/998/status
Name:   process_monitor
Umask:  0022
State:  S (sleeping)
Tgid:   998
Ngid:   0
Pid:    998
PPid:   322
TracerPid:      0
Uid:    1000    1000    1000    1000
Gid:    1000    1000    1000    1000
FDSize: 256
Groups: 4 24 27 30 46 100 1000
NStgid: 998
NSpid:  998
NSpgid: 998
NSsid:  322
Kthread:        0
VmPeak:     2768 kB
VmSize:     2768 kB
VmLck:         0 kB
VmPin:         0 kB
VmHWM:      1944 kB
VmRSS:      1944 kB
RssAnon:              96 kB
RssFile:            1848 kB
RssShmem:              0 kB
```

3. Using top

```
amrutha@blue:~$ top
top - 15:33:11 up 46 min,  1 user,  load average: 0.02, 0.01, 0.00
Tasks:  30 total,   1 running,  29 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.1 sy,  0.0 ni, 99.9 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   7552.6 total,   6882.0 free,    533.8 used,    287.2 buff/cache
MiB Swap:   2048.0 total,   2048.0 free,      0.0 used.   7018.8 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
     87 root      20   0   35116  12376   9296 S   0.7   0.2   0:09.61 systemd-udevd
      1 root      20   0   24256  15472  11644 S   0.0   0.2   0:01.25 systemd
      2 root      20   0    3180   2208   2072 S   0.0   0.0   0:00.01 init-systemd(Ub
      6 root      20   0    3180   2048   1940 S   0.0   0.0   0:00.00 init
     46 root      19  -1   50392  16940  15644 S   0.0   0.2   0:00.41 systemd-journal
     81 systemd+  20   0   22540  14628  12008 S   0.0   0.2   0:00.11 systemd-resolve
    152 root      20   0    2888   2016   1884 S   0.0   0.0   0:00.07 chronyd-starter
    155 root      20   0    4472   3076   2832 S   0.0   0.0   0:00.02 cron
    156 message+  20   0    8820   5364   4692 S   0.0   0.1   0:00.09 dbus-daemon
    163 root      20   0   44092  29784  14640 S   0.0   0.4   0:00.25 networkd-dispat
    180 root      20   0   18432   9476   8236 S   0.0   0.1   0:00.13 systemd-logind
    185 syslog    20   0  220548   5992   4696 S   0.0   0.1   0:00.12 rsyslogd
    241 _chrony   20   0   24252  11804   7220 S   0.0   0.2   0:00.19 chronyd
    255 root      20   0  123456  32488  17872 S   0.0   0.4   0:00.22 unattended-upgr
    262 root      20   0    5492   2796   2612 S   0.0   0.0   0:00.00 agetty
    266 _chrony   20   0   12092   2520   1876 S   0.0   0.0   0:00.00 chronyd
    318 root      20   0    3184   1112    980 S   0.0   0.0   0:00.00 SessionLeader
    319 root      20   0    3200   1248   1108 S   0.0   0.0   0:00.17 Relay(322)
    322 amrutha   20   0    6268   5596   3928 S   0.0   0.1   0:00.15 bash
    324 root      20   0    8644   5572   4732 S   0.0   0.1   0:00.02 login
    390 amrutha   20   0   22500  13500  11048 S   0.0   0.2   0:00.19 systemd
    392 amrutha   20   0   22908   4072   2072 S   0.0   0.1   0:00.00 (sd-pam)
    421 amrutha   20   0    6136   5568   3992 S   0.0   0.1   0:00.04 bash
    777 root      20   0    3184   1120    980 S   0.0   0.0   0:00.00 SessionLeader
    778 root      20   0    3200   1256   1108 S   0.0   0.0   0:00.05 Relay(783)
    783 amrutha   20   0    6268   5604   3936 S   0.0   0.1   0:00.15 bash
    998 amrutha   20   0    2768   1944   1848 S   0.0   0.0   0:00.00 process_monitor
   1025 amrutha   20   0    8172   6064   3956 R   0.0   0.1   0:00.02 top
   1026 root      20   0   35120   6504   3392 S   0.0   0.1   0:00.00 (udev-worker)
   1027 root      20   0   35120   6504   3392 S   0.0   0.1   0:00.00 (udev-worker)
```

4. After Termination

```
amrutha@blue:~$ ps -p 998
    PID TTY          TIME CMD
```

Result

The process was successfully created, monitored using ps, top and /proc, and finally terminated. The required process states were observed during the experiment.